

Day 2 — Learning Content

Timers, Counters & Sequencing — full teaching script

1. Timers — TON and TOF (35 min)

Opening framing (say this to the class):

"Yesterday, everything happened instantly — press a button, something reacts immediately. Today we introduce time. A timer lets the PLC wait before doing something, or keep doing something for a while after a trigger. This is how machines get delays, hold periods, and countdowns."

1.1 — TON (Timer On-Delay)

Explain the rule first, then prove it with an example — don't reverse this order:

Rule: A TON timer starts counting the moment its input goes ON. If the input stays ON continuously until the preset time is reached, the timer's output turns ON. If the input turns OFF before reaching the preset, the elapsed time resets to zero immediately — it does not pause and resume, it restarts from scratch.

Worked example — walk through this exact scenario on the whiteboard before touching the software:

Preset = 3 seconds. A student presses a button and holds it.

- 1 At 0 sec: button pressed, timer starts counting: 0.0 → 0.1 → 0.2 ...
- 2 At 1.5 sec: student releases the button. Timer immediately resets to 0.0. Output never turned ON.
- 3 Student presses again and holds continuously this time.
- 4 At 3.0 sec: preset reached. Output turns ON. It stays ON as long as the button stays held.
- 5 Student releases the button. Output turns OFF immediately (this part is instant, not delayed — only the turning-ON was delayed).

This is the single most common point of confusion: students expect the timer to "remember" the 1.5 seconds from the first attempt and continue from there on the second press. It does not. A standard TON always restarts from zero if the input drops, even briefly. Say this explicitly, then let them verify it themselves in the next exercise rather than just taking your word for it.

1.2 — TOF (Timer Off-Delay)

Rule: A TOF timer's output turns ON immediately the instant its input turns ON — no delay at all on the way in. The delay happens on the way out: when the input turns OFF, the output stays ON for the preset duration before finally switching OFF.

Worked example:

Preset = 3 seconds. Represents a fan that should keep clearing air for 3 seconds after being switched off.

- 1 At 0 sec: switch turned ON. Fan output turns ON immediately.
- 2 At 5 sec: switch turned OFF. Fan output stays ON, timer starts counting down.

3 At 8 sec (5 + 3): preset reached. Fan output finally turns OFF.

Ask the class directly: "If I turn the switch back ON at 6 seconds — one second after switching off, in the middle of that 3-second tail — what happens?" Let a few students guess before answering. The correct answer: the fan output was already ON (it never turned off), and the pending OFF-delay is cancelled — the fan simply continues running as if it was never switched off. This question reliably separates students who have the mental model right from those still guessing.

1.3 — RTO (Retentive Timer On) — brief mention

A normal TON forgets its progress the moment the input drops (as shown above). An RTO does not — it keeps its accumulated time even across multiple ON/OFF cycles, and only resets when a separate Reset input is triggered. This matters for tracking cumulative run-time, e.g., "this motor has run for a total of 8 hours since the last service reset," even if it was started and stopped many times in between. Keep this brief — one example is enough, this is context for later, not something to build a full exercise around today.

1.4 — Hands-on: build all three, in this order

- 1 Build TON first: Button → 3 sec TON → Lamp. Test the "release early, resets to zero" behavior explicitly — this is the point of the exercise, not just getting the lamp to light.
- 2 Build TOF second, on a separate lamp: Button → Lamp turns ON instantly. Release → Lamp stays on 3 more seconds. Test the "press again during the tail" scenario from the worked example above.
- 3 If the software supports it, build RTO third: trigger it in three short 1-second bursts with gaps between them, and confirm the accumulated time is roughly 3 seconds total, not reset to zero between bursts.

Checkpoint before moving on — do not skip this: ask one student to explain, without looking at the screen, which timer type they'd use for (1) a conveyor that should start 2 seconds after a start button is pressed, and (2) a workshop exhaust fan that should keep running 30 seconds after the machine is switched off. If they can answer both correctly and explain why, the concept has landed. If not, redo the worked examples above rather than moving forward.

2. Counters — CTU and CTD (30 min)

Opening framing (say this to the class):

"Timers measure time. Counters measure events — how many times something has happened. Anywhere you need to count parts, cycles, or repetitions, this is the tool."

2.1 — CTU (Count Up)

Rule: A CTU counter increases its current count by exactly 1 each time its count input transitions from OFF to ON — this is called a "pulse" or a "rising edge." It does NOT keep counting while the input is held ON; it only counts the moment of transition. Once the current count reaches the preset value, the counter's output turns ON and stays ON, even if more pulses arrive afterward, until it's reset.

Worked example:

Preset = 5. A push-button stands in for a part sensor on a conveyor.

- 1 Press 1: count becomes 1. Output stays OFF ($1 < 5$).
- 2 Press 2, 3, 4: count becomes 2, 3, 4. Output still OFF.
- 3 Press 5: count becomes 5. Output turns ON.
- 4 Press 6 (extra press): count may continue to 6 depending on the software, but output remains ON — it doesn't turn back OFF on its own.
- 5 Reset input triggered: count returns to 0. Output turns OFF.

2.2 — CTD (Count Down)

Rule: A CTD counter starts at a preset value and decreases by 1 with each pulse on its count-down input. Once the count reaches exactly 0, the output turns ON.

Real-world framing: CTD suits countdown-style logic — for example, "5 cycles remaining before scheduled maintenance," decreasing with every completed cycle, with the output triggering a maintenance-due alarm once it hits zero.

2.3 — Hands-on exercise

- 1 Build a CTU with preset 5 using a push-button as the pulse input
- 2 Confirm the output only turns ON exactly on the 5th press, not before
- 3 Wire a second button to the counter's Reset input, confirm it returns the count to 0
- 4 Extra practice: build a CTD starting at 5, confirm the output turns ON only once it reaches exactly 0, not before

The most common mistake at this stage is students expecting the counter to keep incrementing continuously while the button is held down, the way a light might stay on while a button is held. It doesn't — it only counts the transition from not-pressed to pressed, once per press. If a student's count is climbing rapidly while holding the button, this is almost always because the software is reading the held signal as many rapid pulses rather than one clean edge — worth demonstrating the difference between a quick tap and a long hold side by side so they can see the distinction directly on screen.

Checkpoint: ask a student to predict, before testing, what the count will show after exactly 3 taps on a CTU with preset 5, and whether the output will be ON or OFF at that point. Correct answer: count = 3, output OFF. This is a good gut-check for whether "preset" and "current count" are understood as two separate things.

3. Combine Timer + Counter, Group Troubleshooting (30 min)

Why this exercise exists: timers and counters each seem simple in isolation, but real bugs happen when they're combined — and this is deliberately the first place in the course where students are asked to debug something themselves rather than follow a prescribed build. Resist the urge to solve it for them immediately when something goes wrong; that's the actual point of this block.

The scenario to set up for the class:

"Imagine your part sensor is a bit jittery — sometimes it triggers for a fraction of a second due to vibration, even when no real part is there. We want to only count a part if the sensor stays triggered for at least 2 full seconds — a genuine part, not a false flicker."

3.1 — Building it, step by step

- 1 Step 1: wire the button (standing in for the sensor) into a TON timer with preset 2 seconds.
- 2 Step 2: wire the TON's OUTPUT (not the button directly) into the counter's count-up pulse input.
- 3 Step 3: test with a quick tap — under 2 seconds. The count should NOT increase.
- 4 Step 4: test by holding for over 2 seconds. The count SHOULD increase by exactly 1.

3.2 — What typically goes wrong, and how to guide students through it

Rather than listing these as things to avoid, treat them as expected discoveries — walk between pairs and ask leading questions rather than pointing at the fix directly.

- **Symptom: nothing counts, ever, no matter how long the button is held.** Ask: "what is actually wired into the counter's pulse input right now — the button, or the timer's output?" Students very often wire the counter directly to the original button out of habit, forgetting to redirect it through the timer.
- **Symptom: the count jumps by more than 1 per hold.** Ask: "is the timer's output only ON for one scan, or does it stay ON the whole time the button is held?" If the timer's output is used directly and stays ON continuously once triggered, the counter may see it as a sustained signal rather than a single clean pulse, depending on how the specific software handles counter inputs — this is worth demonstrating live if it comes up, since the exact behavior varies by platform.
- **Symptom: the count resets unexpectedly.** Ask: "is anything else in your rungs wired to the counter's reset input by accident?" A stray wire to reset is a common copy-paste mistake when reusing rungs from the earlier CTU exercise.

Checkpoint: before moving to the break, have every pair demonstrate both the "quick tap does nothing" and "2+ second hold counts once" behaviors live. Don't accept a partial demonstration — both cases need to be shown, since it's easy to get one working while accidentally breaking the other.

— BREAK — 15 min —

4. Day Project: Traffic Light Controller (115 min)

Framing for the class before they start building:

"This project uses only what you already know — timers, sequencing, and turning things on and off. Nothing new is being introduced. What's new is that you're building all of it yourselves, from a plain description, without a step-by-step recipe."

4.1 — The specification to give students

Write this on the board exactly, and don't pre-solve it for them:

- 1 Red ON for 5 seconds, then switches to Green
- 2 Green ON for 5 seconds, then switches to Yellow
- 3 Yellow ON for 2 seconds, then switches back to Red
- 4 This repeats forever once started, with no manual intervention needed between cycles

4.2 — The core idea to teach before they start building

This is the one genuinely new idea in the project, even though it uses only timers: **a timer's own output can be used to trigger the next stage, and to turn off the current stage**. This "handoff" pattern — one stage's completion becoming the next stage's trigger — is worth demonstrating on the whiteboard with a simple two-stage example before students touch the software.

Simplified two-stage whiteboard example (Red → Green only, ignore Yellow for this demo):

```
-| Start |—————( Red )—  
-| Red  |—| TON 5s |—( Green )—  
-| Green |—————|/ Red |—
```

Read this aloud, rung by rung: "Starting turns Red on. Once Red has been on for 5 seconds, Green turns on. And separately — this is the part people forget — once Green is on, it needs to actively turn Red back OFF, using a normally-closed Green contact in Red's own rung. Without that last piece, both lights would stay on together."

This is worth emphasizing explicitly: students often build the "turn the next stage on" logic correctly on their first attempt, but forget the "turn the previous stage off" half entirely, because the specification only describes what should turn on, not what should turn off. Ask them directly: "for every stage that turns on, what is responsible for turning the previous one off?" before they start wiring the full 3-stage version.

4.3 — Full 3-stage build sequence

- 1 Wire or simulate all 3 lamps first: Red, Yellow, Green — confirm all three can be turned on/off manually before adding any sequencing logic, just to rule out wiring issues early.
- 2 Build the Red stage alone: a Start input turns Red on. Test it in isolation.
- 3 Add the Red-to-Green handoff: Red's own 5-second TON output turns Green on AND turns Red off. Test that Red turns off exactly when Green turns on — not before, not after, and not both staying on together even briefly.
- 4 Add the Green-to-Yellow handoff: Green's own 5-second TON output turns Yellow on AND turns Green off. Test the same way.
- 5 Add the Yellow-to-Red handoff: Yellow's own 2-second TON output turns Red back on AND turns Yellow off — this is the one that closes the loop back to the start.
- 6 Let it run for at least 3 full loops uninterrupted, watching for the cycle to repeat cleanly rather than freezing or skipping a stage.

4.4 — Debugging guidance for common failures

- **Sequence runs once, then freezes on Yellow.** Ask: "what rung is supposed to turn Red back on once Yellow finishes?" This is almost always a missing or incorrectly wired final handoff back to Red.

- **Two lights are on briefly at the same time.** Ask: "in the rung for the light that should be turning off, what condition is actually making it turn off?" Usually the "turn off" half of a handoff was never wired, or was wired to the wrong timer.
- **Timing feels wrong (lights change too fast or too slow).** Ask them to check the actual preset value entered on each timer, not just assume it matches the spec — a simple typo (0.5 instead of 5) is common and easy to overlook visually.

4.5 — Extra challenge, for pairs who finish early

- Add a pedestrian-crossing button that forces an early transition from Green to Yellow, rather than waiting the full 5 seconds — this requires OR-ing the button into the same condition that currently only comes from the timer.
- Add a separate "flashing yellow" caution mode, triggered by a dedicated switch, that overrides the normal Red-Green-Yellow sequence entirely with a rapid on/off blink on the Yellow lamp only.

End of day checkpoint: every student/pair demonstrates the Traffic Light Controller running at least 3 full loops without manual intervention, and can explain out loud what would happen if the Yellow-to-Red handoff rung were deleted, before leaving.